GitHub Insights

Streaming Analytics for GitHub Repository Trend Detection

Jann Erhardt
erharjan@students.zhaw.ch
Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Gian Gamper
gampegia@students.zhaw.ch
Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Jonas Bratschi
bratsjon@students.zhaw.ch
Zurich University of Applied Sciences
Winterthur, Zurich, Switzerland

Abstract

GitHub's public event stream exposes a continuous feed of repository activity spanning commits, pull requests, forks, and releases. Identifying early-stage projects with genuine momentum from this feed is impractical without automated analysis. This paper presents *GitHub Insights*, a streaming analytics platform that continuously ingests the GitHub public events API, enriches the associated user and repository records with geographic and profile metadata, and surfaces statistically significant growth trends through a web dashboard and REST API.

The system is built on Apache Kafka (KRaft mode) for durable event buffering, TimescaleDB for compressed hypertable storage with pre-computed continuous aggregates, FastAPI for a REST and Server-Sent Events interface, and a Next.js 16 dashboard. A central component is a *Hidden Gem discovery engine* that assigns a Poisson significance score to repositories, users, and organisations based on observed star and fork activity relative to each entity's historical baseline, across three configurable time windows (24 h, 168 h, 730 h). Over the evaluation period the pipeline ingested over 21 million events covering more than 5 million distinct repositories and 217 countries at a 98.0 % geocoding success rate.

Keywords

stream processing, GitHub event API, Apache Kafka, TimescaleDB, streaming analytics, open-source trend detection, geo-enrichment, Server-Sent Events, Next.js dashboard

1 Introduction

The open-source ecosystem hosted on GitHub grows continuously, with millions of push events, pull requests, issue comments, and repository forks published every day [2]. For developers, researchers, and technology scouts this volume presents a paradox: the very richness of the signal makes it practically impossible to manually track emerging projects before they appear in mainstream media.

Existing discovery mechanisms rely on lagging indicators (stars accumulated over weeks, blog posts, or curated lists) rather than on the raw momentum visible in the live event stream. A repository that doubles its commit frequency or attracts contributors from three continents in a single day represents a signal of genuine innovation; today that signal is largely wasted.

This project addresses the gap with *GitHub Insights*: an end-to-end streaming analytics platform that continuously ingests the GitHub public events API, enriches the associated user and repository records with geographic and profile metadata, persists the data in a time-series optimised store, and surfaces trends through an interactive web dashboard. A central component is a *Hidden Gem discovery engine* that assigns a statistical significance score to each repository based on observed star and fork activity relative to its historical baseline, applying a Poisson significance test across three time windows (24 h, 168 h, 730 h).

The remainder of this paper is structured as follows. Section 2 surveys related approaches. Section 3 describes the system architecture. Section 4 details the implementation of each component. Section 5 presents evaluation results. Section 6 justifies the technology choices. Section 7 discusses challenges encountered. Section 8 concludes with lessons learned and future work.

2 Related Work

Detecting emerging open-source projects before they gain widespread attention requires both historical context and the ability to process high-velocity event streams. Several prior systems address these challenges from complementary angles.

GitHub Archive and GHTorrent. GH Archive [5] pioneered the idea of recording GitHub's public timeline by crawling the `GET /events` endpoint hourly and publishing the raw JSON payloads as gzip-compressed files on Google Cloud Storage, enabling retrospective analyses of GitHub's growth and contributor behaviour over multiple years [7]. Live data collection for GH Archive was subsequently discontinued; the dataset remains accessible as a historical archive via Google BigQuery. GHTorrent [3] extended this concept by mirroring events and supplementing them with full dumps of repository and user metadata from the GitHub REST API. GHTorrent was officially discontinued in 2019 following changes to GitHub's API access policies. While both systems provided valuable historical depth, neither offered near-real-time ingestion or pre-computed analytics.

Open-source trend detection. GitHub's own Trending page surfaces repositories by star velocity over a configurable window, but is limited to stars and does not expose the underlying event stream or normalise velocity against a repository's history. Similar read-only services exist but provide no ingest or enrichment capabilities. The Hidden Gem engine introduced in this work applies a Poisson significance test that normalises observed growth against each repository's historical baseline, enabling statistically grounded comparison across repositories of different ages and popularity levels, a distinction not found in existing publicly documented tools.

PM4 FS26, ZHAW, Winterthur, Switzerland
2026.

3 System Architecture

Figure 1 shows the end-to-end architecture: a streaming pipeline designed to continuously ingest, enrich, and expose GitHub event data. Individual components are decoupled through a message broker to allow independent scaling and failure isolation. Table 1 lists all Docker Compose services.

Table 1: Docker Compose services and their roles

Service	Function
`producer`	Polls the GitHub API; writes raw events to Kafka.
`kafka`	Central message broker for decoupling and buffering.
`kafka-ui`	Web UI (port 8080) for topic inspection and lag monitoring.
`kafka-init`	One-shot container that creates Kafka topics on first start.
`consumer-{0,1,2}`	Three parallel enrichment instances, each pinned to one Kafka partition.
`geocoder`	Claims unlabelled user locations and resolves them via Photon.
`photon`	Local OpenStreetMap geocoding engine; resolves location strings to coordinates.
`db-writer`	Consumes status and rate-limit topics; persists operational metrics.
`timescaledb`	Time-series optimised SQL database for event storage and aggregation.
`api`	FastAPI service exposing REST and SSE endpoints (port 8000); runs the snapshot scheduler.
`frontend`	Next.js dashboard serving the end-user web interface (port 3000).
`grafana` [4]	Operational monitoring dashboards for pipeline health and system metrics.

3.1 Data Ingestion (Producer)

The **Producer** (`producer/producer.py`) serves as the entry point of the pipeline:

- **Polling:** It polls the public GitHub Events API every 10 seconds.
- **Deduplication:** To prevent redundant data, the producer filters events based on their unique GitHub Event ID.
- **Kafka Integration:** Cleaned raw JSON data is published to the Kafka topic `github.events.raw`.
- **Authentication:** A single GitHub personal access token raises the API rate limit from 60 to 5 000 requests per hour [2], which is sufficient for the 10-second poll cycle. The producer uses one token exclusively for polling; enrichment tokens are managed separately by the consumer.

3.2 Message Broker (Kafka)

Kafka [1] acts as the central message bus, operating in KRaft mode without the need for Zookeeper:

- **Decoupling:** It separates the high-frequency data ingestion (Producer) from the more complex data enrichment steps (Consumer).
- **Durability:** Kafka serves as a reliable buffer; if a consumer fails, data is retained for 48 hours, allowing the system to resume where it left off.
- **Scalability:** The main topics are divided into three partitions, enabling parallel processing by multiple consumer instances.
- **Topics:** Four topics partition the payload types: raw event JSON, enrichment status, rate-limit snapshots, and geocoder request logs (single partition).

3.3 Processing and Enrichment (Consumer & Geocoder)

The consumer transforms raw Kafka messages into structured, enriched records by querying the GitHub API for user and repository metadata. Enrichment is bounded by GitHub's rate limits, and several mechanisms are used to operate within those constraints.

- **Concurrent Consumer Instances:** The consumer supports a configurable multi-instance mode in which each running instance is statically assigned a disjoint subset of Kafka partitions (`KAFKA_INSTANCE_INDEX`, `KAFKA_TOTAL_INSTANCES`). In the evaluation setup, three instances were deployed, one per partition, to process events in parallel without overlap.
- **Token Pool Management:** Multiple GitHub personal access tokens are pooled separately for user/organisation and repository enrichment. The pool rotates across tokens in round-robin order and temporarily skips any token that has been rate-limited, resuming it after its reset timestamp has passed.
- **GraphQL Batch Enrichment:** Metadata is fetched via the GitHub GraphQL API in batches of up to 20 entities per request, reducing the number of API calls needed relative to individual REST lookups.
- **REST Fallback:** If a GraphQL response returns `null` for a specific entity, the consumer issues an individual REST API call for that entity as a fallback.
- **Geocoding:** A dedicated `geocoder` service processes user location strings asynchronously. It claims unresolved rows from the database and queries the local Photon instance, decoupling geographic resolution from the main event-processing loop.

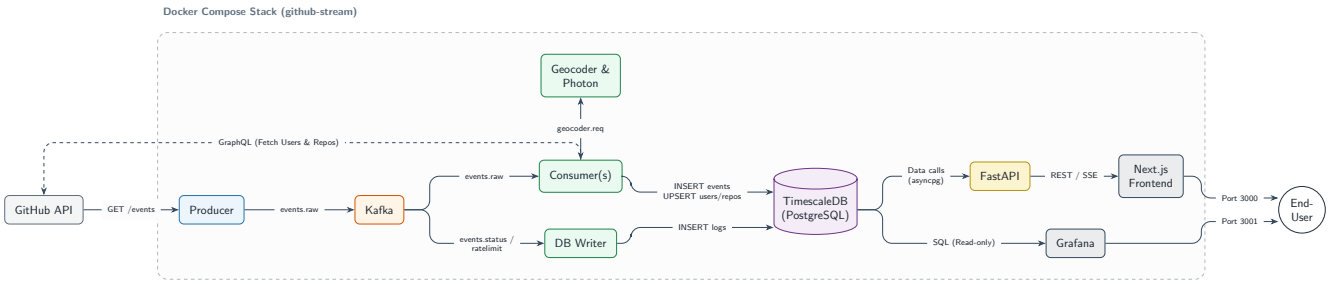

Figure 1: Simplified end-to-end architecture of GitHub Insights. Events flow from the GitHub API through Kafka (KRaft) into TimescaleDB, where they are served by FastAPI to the Next.js dashboard and Grafana. Dashed arrows indicate auxiliary data flows. See Figure 2 in Appendix A for the full annotated diagram with all port numbers and data-flow labels.

3.4 Data Storage (TimescaleDB)

The project uses **TimescaleDB** [10], an open-source time-series database built as a PostgreSQL extension, as its primary storage solution:

- **Hypertables:** The `events` table is declared as a hypertable and automatically partitioned into time-based chunks of one day each. Chunks older than seven days are compressed using columnar storage, reducing storage requirements significantly (see Section 5).
- **Continuous Aggregates:** Two pre-computed rollup views (5-minute event counts and country-code counts) materialise at ingest time, reducing query latency for time-range aggregations compared to scanning the full hypertable.

The relational schema is organised into three groups: the core star schema (Figures 3), the Hidden Gem snapshot tables (Figure 4), and the operational log tables (Figure 5), all documented in Appendix C.

3.5 Visualisation

The system provides two complementary visualisation layers, each targeting a different audience and purpose.

Next.js Frontend (port 3000). The primary end-user interface provides near-real-time analytics and trend discovery. An SSE-based event feed polls new records from TimescaleDB every three seconds, displaying recently ingested events with a corresponding propagation delay. Server-side components load KPI summaries, activity leaderboards, and geographic heatmaps at request time. The Hidden Gems section exposes scored repositories, users, and organisations across three time windows, with filter controls and detail pages derived from the snapshot tables.

Grafana (port 3001). Grafana serves as the operational monitoring layer for system operators. Seven provisioned dashboards cover three areas:

- **Pipeline overview:** Aggregate event counts, geographic coverage, and ingestion statistics.
- **API monitoring:** Response-time distributions and request volumes for REST and GraphQL calls separately, and per-token rate-limit snapshots.
- **Geocoder monitoring:** Geocoding request throughput, latency, and error rates.

These dashboards are intended for operational inspection and are not part of the end-user discovery workflow.

3.6 Geocoding Engine (Photon)

To implement high-performance geographic enrichment of GitHub events without relying on external rate-limited services, **Photon** was integrated into the infrastructure.

- **Local Instance:** Photon is an open-source geocoding service based on OpenStreetMap data. By operating it as a local Docker container, latency is minimized and the strict usage constraints of public APIs (such as Nominatim [8]) are bypassed.
- **Data Basis:** The production deployment uses the global OpenStreetMap planet dataset, persisted in the `photon_data` Docker volume. Smaller regional subsets are supported for development environments.
- **API Interface:** Photon provides a REST API that is queried by the `geocoder` service. It processes unstructured location strings from GitHub profiles and returns structured JSON data, including latitude, longitude, and ISO country codes.
- **Decoupled Enrichment:** The geocoder runs as a separate service that claims unresolved location strings from the database independently of the consumer. A failure or restart of the geocoder does not block event ingestion; unresolved locations are retried on the next run.

3.7 Trending Repository Discovery

The trending repository discovery module operates as an analytical layer on top of the enriched event data stored in TimescaleDB. It consumes the `events` and `repos` tables produced by the consumer and geocoding services and exposes its results through the FastAPI layer and the Next.js frontend.

The module comprises three components. First, a set of three PostgreSQL aggregate functions (one each for repositories, users, and organisations) compute per-entity significance scores on demand from the raw event tables. Second, a snapshot scheduler materialises these scores into persistent snapshot tables at configurable intervals (24 h, 168 h, 730 h). Third, a dedicated FastAPI sub-router exposes filtered rankings, entity detail views, and cohort evaluation endpoints.

This design keeps scoring logic entirely within the database tier, where TimescaleDB can leverage time-based indexing and column compression. No additional microservice is required; the scheduler runs as a background thread within the FastAPI container.

4 Implementation

This section details the concrete implementation of each pipeline component, focusing on decisions that are not fully captured by the architectural overview: polling and deduplication logic, enrichment batching, storage configuration, the API and frontend design, and the Hidden Gem scoring engine.

4.1 Data Ingestion

The producer (`producer/producer.py`) requests up to three pages of 30 events each from the `GET /events` endpoint per poll cycle (90 events at most per cycle). Because successive poll windows overlap the GitHub API's server-side cache, an in-process set of recently seen event IDs filters out duplicates before messages are forwarded to Kafka. The net number of unique new events per cycle depends on GitHub's actual publishing rate; throughput measurements are reported in Section 5.

4.2 Event Enrichment

The consumer (`consumer/consumer.py`) feeds two independent batch queues inside an `Enricher` instance, each backed by its own GitHub token pool:

(1) **User/organisation-profile enrichment.** The login names of event actors (the GitHub users who triggered each event) are enqueued in a user/org batch queue (up to 20 items per GraphQL request). Before enqueueing, the `Enricher` checks for an existing database row with `fetched_at IS NOT NULL`; already-enriched accounts are skipped. If a GraphQL response returns `null` for a specific entity, the consumer falls back to an individual REST call; a 404 response from the REST endpoint marks the entity as permanently absent to avoid repeated enrichment attempts.

(2) **Repository enrichment.** Repository IDs follow the same pattern through a separate batch queue with its own token pool. Resolved metadata (language, topics, star and fork counts) is upserted into the `repos` table. Geographic enrichment of user location strings is delegated to the `geocoder` service (Section 3.6).

Enriched profile data is written to four normalised tables: `users`, `repos`, `organizations`, and `organization_members`. Raw event records are written separately to the `events` hypertable without modification.

4.3 Storage Layer

The `events` table is a TimescaleDB hypertable partitioned into one-day chunks. Chunk-level columnar compression is applied after seven days (sorted by `time DESC`, segmented by `event_type`); the measured compression ratio and its influencing factors are discussed in Section 5. A 90-day data retention policy bounds disk usage.

Two continuous aggregates pre-compute rollups at ingest time rather than query time (Table 2), reducing query latency for time-range aggregations compared to scanning the raw hypertable.

Table 2: TimescaleDB continuous aggregates

View	Bucket	Content
`event_stats_5m`	5 min	Event counts by `event_type`
`country_ids_5m`	5 min	Event counts by ISO country code

4.4 API Layer

The FastAPI [9] service exposes the endpoints listed in Table 3. The `/stream/events` endpoint implements the Server-Sent Events protocol [6]: it queries the `events` table (joined with `repos`) every 3 seconds and writes JSON-encoded event objects to the response stream. An initial `: heartbeat` comment is written immediately to register the connection, followed by `: keep-alive` comments between data frames to prevent browser and proxy timeouts.

4.5 Visualisation

The Next.js [11] frontend (port 3000) uses React server components for request-time data fetching and an SSE client hook for live updates. The main analytics dashboard presents four KPI cards (total commits, pull requests, unique contributors, and average PR review time); a daily commit activity chart; a ranked table of the most active repositories in the preceding 24 hours; a near-real-time event feed; and a choropleth world map with globe projection of event volume by country and an activity leaderboard.

A dedicated *Hidden Gems* section provides a paginated leaderboard switchable between three entity scopes (repositories, users, organisations) and three time windows (24 h, 168 h, 730 h). Repository rankings support attribute filters on programming language, software licence, and topic tags; a global search bar enables cross-entity lookup across all snapshot data. Individual detail pages display each entity's significance score alongside a score-history chart; user pages list scored repositories with individual scores and organisation pages show separate aggregates for directly owned and member-contributed repositories. An *Evaluation Reports* view exposes cohort classification results (see Section 5.5).

Table 3: FastAPI endpoint reference

Endpoint	Description
`GET /health`	Liveness check
`GET /api/kpis`	KPI metrics (commits, PRs, contributors, avg review time)
`GET /api/commits-over-time`	Daily commit buckets (configurable window)
`GET /api/top-repos`	Most active repositories (24 h window)
`GET /api/recent-events`	Latest ingested events
`GET /stream/events`	SSE live stream (3 s poll)
`GET /api/overview/heatmap`	Event counts by country for choropleth map
`GET /api/overview/globe-heatmap`	Globe-projection heatmap data
`GET /api/activity/leaderboard`	Top actors by event count (configurable window)

4.6 Hidden Gem Scoring Engine

4.6.1 Momentum Metric. We define *momentum* for a repository r in a time window $[t - \Delta,\ t]$ as the weighted count of starring and forking events $k = \lfloor \alpha \cdot k_\star + \beta \cdot k_{\text{fork}} \rfloor$ observed in that window, where α and β are configurable coefficients (default: $\alpha = \beta = 1.0$). Raw momentum counts are not directly comparable across repositories: a repository with 50,000 cumulative stars receiving 100 new stars in 24 h reflects qualitatively different behaviour from a two-month-old repository with 300 cumulative stars receiving the same count. The scoring therefore normalises observed growth against an expected rate derived from the full event history of each repository.

4.6.2 Statistical Scoring Model.

Expected rate. Let λ denote the expected weighted event count in the window under a stationary Poisson null model:

$$\lambda = \frac{\alpha \cdot S_{\text{base}} + \beta \cdot F_{\text{base}}}{\max(A_r,\ 1)} \times \min(\Delta,\ A_r)$$

where S_{base} and F_{base} are the cumulative star and fork counts accrued before the window, A_r is the repository age in hours at the start of the window, and Δ is the window duration in hours. For repositories younger than the window ($A_r \leq \Delta$), the factor $\min(\Delta, A_r) = A_r$ cancels the denominator, reducing λ to the full historical baseline. This prevents repositories with limited history from receiving inflated scores due to a short observation period.

Significance score. Let $X \sim \text{Poisson}(\lambda)$ and k be the observed weighted event count. The Poisson survival function is $\text{SF}(k) = P(X \geq k;\ \lambda)$. The significance score is defined as:

$$\text{sig\_score} = \min\left(\frac{-\ln \text{SF}(k)}{34.5} \times 10,\quad 10\right)$$

The score equals zero when k is consistent with the expected rate and grows as observed growth becomes increasingly improbable under the null model, with a ceiling of 10. The constant 34.5 normalises the dynamic range to this maximum. A score of 3.0 corresponds to $\text{SF} \leq e^{-10.35} \approx 3.2 \times 10^{-5}$, the primary classification boundary used by the frontend to designate a repository as a genuine hidden gem. The survival function is computed in log-space via an iterative accumulation scheme (`F_poisson_sf()` in `004_hidden_gem_functions.sql`) to avoid numerical underflow for large k or small λ.

Noise guards. Repositories must satisfy minimum activity thresholds within the window (default: $k_\star \geq 5$, $k_{\text{fork}} \geq 1$) before a score is assigned. Repositories that do not meet these guards receive a `NULL` score and are excluded from rankings. The thresholds are configurable via environment variables.

4.6.3 User and Organisation Aggregation. Significance scores aggregate from repositories to users and organisations. For each user, three metrics are computed from the repositories active in the window: the total score sum, the highest single-repository score, and the count of repositories exceeding sig_score $\geq$ 3.0. For organisations the aggregation produces two independent sub-totals: one for repositories directly owned by the organisation, and one for repositories owned by registered members (resolved via the `organization_members` table). This split allows an organisation to assess its own output and the collective contribution of its member community independently.

4.6.4 Snapshot Scheduler. Because the aggregate queries scan the full `events` table, on-demand execution would place a significant load on the database under concurrent user requests. Results are therefore materialised at fixed intervals by a background scheduler (`snapshot_scheduler.py`), implemented with APScheduler and configured for three intervals: 24 h, 168 h, and 730 h. Stored snapshots serve two purposes: they provide the score history consumed by the frontend detail pages, and they enable the cohort evaluation described in Section 5.5. The snapshot table schema is documented in Appendix C (Figure 4).

4.6.5 Hidden Gem API Endpoints. A dedicated FastAPI sub-router exposes eleven endpoints under the `/api/hidden-gems/` prefix, complementing the general API described in Section 4. Live ranking endpoints carry a 30-second response cache to amortise aggregate query cost; filter metadata endpoints use a 5-minute cache. A restricted trigger endpoint (`POST /snapshots/trigger`) allows authorised clients to initiate an on-demand snapshot run outside the regular schedule.

5 Evaluation

5.1 Experimental Setup

5.1.1 Hardware and Operating System. The entire pipeline was hosted on a dedicated virtualised server running Ubuntu 24.04.4 LTS. System specifications are listed in Table 4.

Table 4: Evaluation server specifications

Component	Specification
Operating System	Ubuntu 24.04.4 LTS (Noble Numbat)
Kernel	Linux 6.17.0-23-generic (x86_64)
CPU	AMD EPYC-Milan (16 vCPUs)
Hypervisor	KVM (full virtualisation)
Memory	31 GiB
Storage	348 GB NVMe virtual disk

5.1.2 Software Environment and Containerization. The architecture is fully containerized using **Docker** and managed via **Docker Compose**. This approach ensures service isolation and simplified dependency management. The primary software components and their respective versions are:

- **Message Broker:** Apache Kafka v3.9.0, running in KRaft mode for metadata management without Zookeeper.
- **Database:** TimescaleDB (based on PostgreSQL 16), utilizing hypertables and continuous aggregates for time-series optimization.
- **Geocoding Engine:** Photon (OpenStreetMap-based) running as a local REST service.
- **Runtime Environment:** Python 3.11+ for the Producer, Consumer, and Geocoder microservices.

5.1.3 Pipeline Configuration. The pipeline ran continuously in multi-consumer mode throughout the evaluation period.

- **Consumers:** Three independent consumer instances were deployed, each statically assigned to one Kafka partition to prevent processing overlaps.
- **Resource Allocation:** With 16 vCPUs available, the Kafka broker, TimescaleDB background workers, and enrichment consumers operated concurrently while maintaining a system load average below 1.0 during peak ingestion.
- **Network:** A bridge-driver Docker network with MTU 1442 was used for inter-container communication.

5.2 Throughput and API Response Times

Table 5 summarises the key performance metrics measured during the evaluation period. The API response times refer to calls made by the consumer to the GitHub GraphQL and REST endpoints for enrichment; they do not represent end-to-end pipeline latency from event publication to dashboard display.

5.3 Data Coverage and Enrichment Quality

Table 6 summarises the breadth of data collected over the full evaluation period; all values were obtained by querying the production database at the end of the run.

Table 5: Pipeline throughput and GitHub API response times

Metric	Value	Notes
Events ingested (total)	21,748,215	Over the full 94-day DB range
Average ingestion rate	161 ev/min	Derived from total over 94-day range
Peak ingestion rate	660 ev/min	
Median API resp. time (GraphQL)	0.294 s	1,754,020 requests; p95 = 0.668 s
Median API resp. time (REST)	0.303 s	5,592,045 requests; p95 = 0.551 s
Kafka consumer lag (steady)	0 msgs	All partitions at log-end offset

Table 6: Data coverage over evaluation period

Dimension	Value
Unique repositories	5,367,217
Unique contributors	3,963,190
Countries represented	217
Geocoding success rate	98.0%
Event types observed	`CommitCommentEvent`, `CreateEvent`, `DeleteEvent`, `DiscussionEvent`, `ForkEvent`, `GollumEvent`, `IssueCommentEvent`, `IssuesEvent`, `MemberEvent`, `PublicEvent`, `PullRequestEvent`, `PullRequestReviewCommentEvent`, `PullRequestReviewEvent`, `PushEvent`, `ReleaseEvent`, `WatchEvent` (16 types)
Most active programming lang.	HTML (824,115 repos)

The 98.0% geocoding success rate reflects the combination of Photon's fuzzy-search capability and the optimistic-locking mechanism that prevents concurrent geocoding of the same location string. The 217 countries and all 16 GitHub event types defined in the public API specification were observed, confirming that the collection is not biased towards specific activity types or geographic regions.

5.4 Storage Efficiency

The `events` hypertable currently spans 94 one-day chunks. Of these, 83 chunks (all data older than 7 days) have been compressed; 11 recent chunks remain in uncompressed row-store format. Table 7 summarises the measured storage figures.

Table 7: events hypertable compression statistics

Metric	Value
Total chunks	94
Compressed chunks	83
Uncompressed chunks (recent)	11
Before compression (83 chunks)	12.75 GB
After compression (83 chunks)	4.43 GB
Overall compression ratio	2.9×

The 2.9× overall ratio reflects the mixed column composition: structured fields (`time`, `event_type`, `actor_username`, `repo_id`) compress to a small fraction of their original row-store size (up to 36× on table data alone), while the `payload` JSONB column stored in PostgreSQL TOAST is already partially compressed internally and benefits less from columnar reordering, moderating the aggregate figure.

5.5 Hidden Gem Detection Quality

5.5.1 Cohort Analysis Methodology. To assess detection quality without external ground truth, we apply a cohort evaluation approach. Each repository receiving a non-null significance score in snapshot run i is classified as *sustained* if it achieves sig_score ≥ 1.5 in the immediately following snapshot of the same interval; otherwise it is classified as *dropped*. The threshold of 1.5 is the persistence criterion used by the system, below the primary hidden-gem boundary of 3.0 but still indicating activity that deviates statistically from the null model. This methodology is implemented in the `/api/hidden-gems/snapshots/{id}/cohort` endpoint and exposed in the *Evaluation Reports* view of the frontend dashboard. Sustained detections are interpreted as genuine momentum signals; dropped detections indicate transient activity spikes.

Table 8: Cohort evaluation results by time window. *Flagged* = all repositories with a non-null score in the snapshot run; *Sustained* = sig_score ≥ 1.5 in the next same-interval snapshot; Precision = Sustained / Flagged.

Window	Snaps.	Flagged	Sust.	Drop.	Prec.
24 h	17	64	38	26	59.4%
168 h	10	465	406	59	87.3%
730 h	10	993	888	105	89.4%

5.5.2 Score Distribution Across Snapshots. Table 9 summarises the sig_score distribution across all stored snapshot runs, computed from a one-time query on the snapshot tables. Statistics are computed over all (repository, run) pairs with a non-null score. The *Scored* column counts unique repositories with at least one non-null score across all runs of a given window type.

The distribution is heavily right-skewed in all three windows: the median is 0.00, meaning that more than half of all scored (repository, run) pairs yielded a score of zero. A score of exactly zero arises when a repository passes the minimum activity noise guards ($k_\star \geq 5$, $k_{\text{fork}} \geq 1$) but its observed growth is at or below the Poisson expected rate ($k \lesssim \lambda$), making $\text{SF}(k) \approx 1$ and $-\ln \text{SF}(k) \approx 0$. This is a natural outcome for established repositories where even moderate star counts are expected; the zero score is therefore a valid and meaningful result of the model, not an artefact.

Fewer than 10% of scored entries in any window reach sig_score $\geq$ 3.0 (the primary hidden-gem boundary), and 3–6% reach the ceiling of 10 ($P(X \geq k) \leq e^{-34.5} \approx 10^{-15}$). The prevalence of ceiling-scoring repositories motivates the sustained-trend analysis presented in Section 5.5.3.

Table 9: sig_score distribution across all snapshot runs, by time window. Statistics over all (repository, run) pairs with a non-null score. *Scored* = unique repositories with at least one non-null score.

Win.	Scored	Mean	Med.	p90	≥3.0	= 10
24 h	557	0.71	0.00	2.09	8.8%	3.1%
168 h	5,352	0.59	0.00	1.13	7.0%	2.8%
730 h	9,113	0.89	0.00	2.39	9.5%	6.4%

5.5.3 Sustained Momentum Detections. Because the sig_score ceiling of 10 is reached by 3–6% of scored repositories per window (Table 9), peak score alone is a poor discriminator within the top tier. A more informative measure of genuine discovery quality is *temporal consistency*: how many independent calendar days did the algorithm flag a repository as statistically significant (sig_score $\geq$ 3.0)?

To compute this, we executed a one-time analytical SQL query directly on the production database (not available in the system's live dashboard). The query replayed the Poisson scoring model day-by-day across the full event history, applying the same λ formula and the same `F_poisson_sf()` function as the live engine. S_{base} was reconstructed as the sum of the estimated pre-collection star count (current GitHub total minus our total observed stars) and the running cumulative tally of stars observed before each target day. Table 10 lists the five repositories that the algorithm flagged on the most distinct calendar days with sig_score $\geq$ 3.0.

Table 10: Top-5 repositories by number of calendar days with sig_score ≥ 3.0 over the full evaluation period, computed by replaying the Poisson scoring model on the raw event history. *Avg. score* is the mean over significant days only. `addyosmani/agent-skills` was still being flagged on the last day of the collection period (2026-05-09), indicating ongoing momentum.

Repository	Lang.	Days	Avg. score	Active period	Description
`forrestchang/andrej-karpathy-skills`	–	9	6.04	Apr 12–26	Community-maintained guide to technical skills and learning resources inspired by AI researcher Andrej Karpathy
`NousResearch/hermes-agent`	Python	8	8.81	Apr 8–18	Agentic AI framework from NousResearch, companion to the Hermes large-language-model family
`Fincept-Corporation/FinceptTerminal`	Python	8	7.98	Apr 18–27	Terminal-based financial analytics platform for real-time market data and portfolio tracking
`addyosmani/agent-skills`	Shell	7	9.70	Apr 25–May 9	Curated collection of reusable AI agent skill definitions for agentic coding workflows, by Addy Osmani (Google)
`multica-ai/multica`	TypeScript	6	7.99	Apr 9–18	Multi-agent AI orchestration and coordination platform

6 Technology Decision Rationale

Apache Kafka. Kafka [1] decouples the rate-limited GitHub poller from the slower geo-enrichment step. Its consumer group model allows the enrichment workload to be parallelised by running additional consumer containers without modifying the producer. The durable log acts as a replay buffer: if the consumer crashes mid-processing, it resumes from its last committed offset without data loss.

TimescaleDB over plain PostgreSQL.. Plain PostgreSQL lacks automatic time-based chunk pruning and columnar compression; queries over large event tables become progressively slower as data accumulates. TimescaleDB [10] adds hypertables (transparent partitioning by time), chunk-level columnar compression, and continuous aggregates that materialise rollups at ingest time. It retains full SQL compatibility, which is essential for the JOIN-heavy enrichment queries and JSONB payload access.

FastAPI over Flask/Django. FastAPI's [9] async-first design and native `StreamingResponse` make SSE implementation straightforward. The automatic OpenAPI documentation at `/docs` was valuable during development and testing.

Next.js 16 (App Router). Server components eliminate the waterfall of client-side data fetches on initial page load. The App Router's layout system provides a persistent sidebar without duplicating markup across pages. Tremor's pre-built chart components (AreaChart, BarChart, LineChart) accelerated dashboard development significantly.

7 Challenges and Solutions

GitHub API rate limiting. The GitHub REST API caps unauthenticated requests at 60 per hour, which is insufficient for continuous polling. A single personal access token raised the limit to 5 000 requests per hour; as enrichment demand grew with parallel consumer instances, a second token was added and both rotated in round-robin order. The enrichment strategy was subsequently extended to a hybrid REST/GraphQL approach: user and organisation metadata is now fetched in batches of up to 20 entities per GraphQL request, reducing API calls per enrichment cycle. Separate token sub-pools for user/organisation and repository enrichment allow the pool to skip any rate-limited token and resume it automatically once its reset window expires.

Geocoding at scale. Geographic enrichment of free-form GitHub user location strings required a geocoding backend capable of handling high request volumes without external rate-limit constraints. An initial implementation queried the Nominatim public API [8]; however, the API's strict usage policy (one request per second for public endpoints) produced a growing resolution backlog under the observed ingestion rate, rendering real-time enrichment impractical. The geocoder was consequently replaced with a local Photon instance (Section 3.6), which operates against a local copy of the OpenStreetMap planet dataset and imposes no per-request rate constraint. Redundant requests across concurrent consumer instances were eliminated with an optimistic-locking claim: the geocoder sets `geo_claimed_at = NOW()` via a single atomic `UPDATE . . . WHERE geo_claimed_at IS NULL`, ensuring each location string is resolved exactly once without requiring a distributed lock.

8 Conclusion and Future Work

This work designed and implemented a streaming analytics pipeline for the GitHub public event stream, combining Apache Kafka for message transport, TimescaleDB for time-series storage, and a Next.js frontend for interactive analysis. Over the full evaluation period the pipeline ingested over 21 million events from more than 5 million distinct repositories, with geocoding resolving user locations across 217 countries at a 98.0 % success rate. The Hidden Gem discovery engine applies a Poisson significance model to quantify statistically anomalous repository growth across three time windows (24 h, 168 h, 730 h). Cohort evaluation yielded precision values of 59 % (24 h), 87 % (168 h), and 89 % (730 h), indicating that longer observation windows produce more stable momentum signals. Several directions remain for future work:

- **Stream analytics.** Attaching Apache Flink or Spark Structured Streaming to the `github.events.raw` topic would enable stateful, sub-second anomaly detection as an alternative to the current batch-snapshot approach, without modifying the existing pipeline.
- **Long-term archival.** A Kafka Connect S3 sink writing Parquet files to object storage would enable cost-efficient historical analysis beyond the 90-day TimescaleDB retention window.
- **Alerting integration.** Configuring Grafana alerting rules on the Hidden Gem snapshot tables could proactively notify users when a repository's significance score crosses a predefined threshold, removing the need for manual inspection of the ranking dashboard.

Acknowledgments

The authors thank Jonathan Fürst and Nico Spahni (PM4 module supervisors, ZHAW School of Engineering) for their guidance throughout the project.

References

[1] Apache Software Foundation. 2025. Apache Kafka Documentation. https://kafka.apache.org/documentation/. Accessed: March 2025.
[2] GitHub, Inc. 2025. GitHub REST API Documentation. https://docs.github.com/en/rest. Accessed: March 2025.
[3] Georgios Gousios. 2013. The GHTorrent dataset and tool suite. https://www.ghtorrent.org/. Accessed: March 2025.
[4] Grafana Labs. 2025. Grafana Documentation. https://grafana.com/docs/grafana/latest/. Accessed: March 2025.
[5] Ilya Grigorik. 2012. GH Archive — a project to record the public GitHub timeline, archive it, and make it easily accessible for further analysis. https://www.gharchive.org/. Accessed: March 2025.
[6] Ian Hickson. 2015. Server-Sent Events — W3C Recommendation. https://www.w3.org/TR/eventsource/. W3C Recommendation, 3 February 2015.
[7] Filipe Lamarque et al. 2022. An analysis of GitHub ecosystem using GH Archive data. *Empirical Software Engineering* 27, 4 (2022), 1–34.
[8] OpenStreetMap contributors. 2025. Nominatim Search API Documentation. https://nominatim.org/release-docs/latest/api/Search/. Accessed: March 2025.
[9] Sebastián Ramírez. 2025. FastAPI Documentation. https://fastapi.tiangolo.com/. Accessed: March 2025.
[10] Timescale, Inc. 2025. TimescaleDB Documentation. https://docs.timescale.com/. Accessed: March 2025.
[11] Vercel, Inc. 2025. Next.js Documentation. https://nextjs.org/docs. Accessed: March 2025.

Generative AI Tool Directory

Anthropic. (2026). *Claude Sonnet 4.6* (Version `claude-sonnet-4-6`). https://www.anthropic.com/claude

- Paraphrasing of text passages
- Structural guidance for the report outline
- Review and debugging of the report document

A Full System Architecture

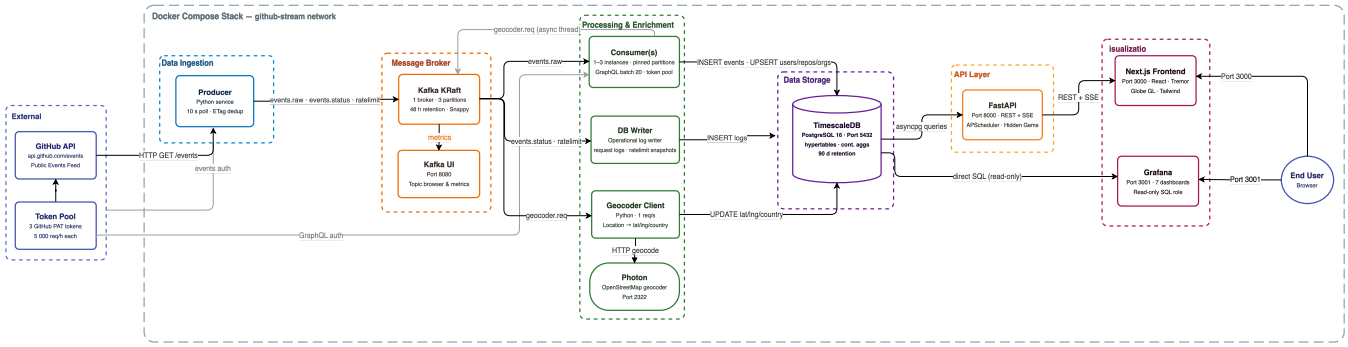

Figure 2: Full annotated system architecture diagram. All seven layers are shown with explicit data-flow labels, port numbers, and service boundaries of the Docker Compose stack.

B Project Timeline

Table 11: Project timeline overview

Week	Phase	Deliverable
KW 03	Setup	Kafka pipeline, TimescaleDB schema, Docker Compose skeleton
KW 04	Exploration	GitHub event stream collection, EDA, schema refinement
KW 06	Pipeline	Producer, consumer, geo-enrichment, FastAPI endpoints
KW 10	Dashboard	Next.js dashboard, Grafana provisioning, SSE live feed
KW 13	Finalization	Testing, documentation, final report, presentation

C Database Schema

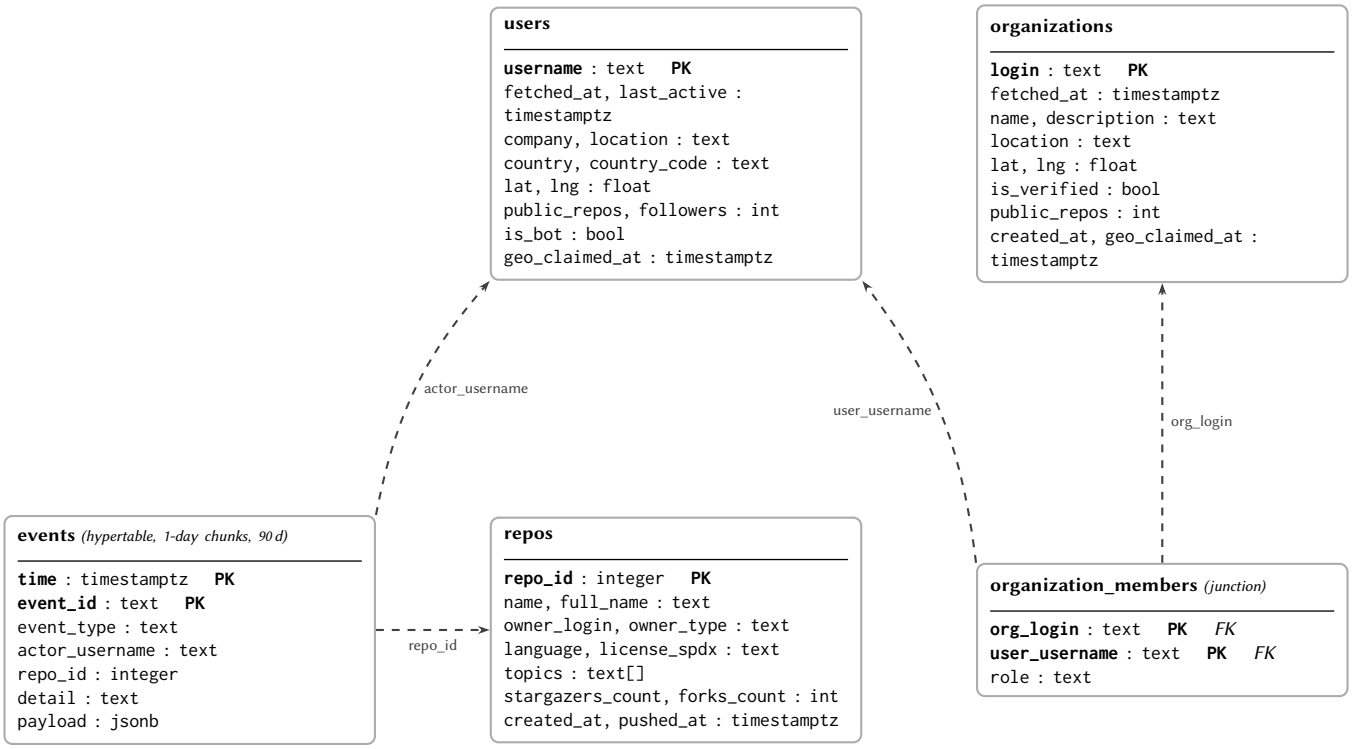

Figure 3: Core star schema. `events` is the central fact hypertable; `users`, `repos`, and `organizations` are dimension tables; `organization_members` is a junction table with enforced `REFERENCES` constraints. Dashed arrows from `events` to `users` and `repos` represent logical joins (no database-level FK constraint); arrows from `organization_members` represent actual `REFERENCES` constraints.

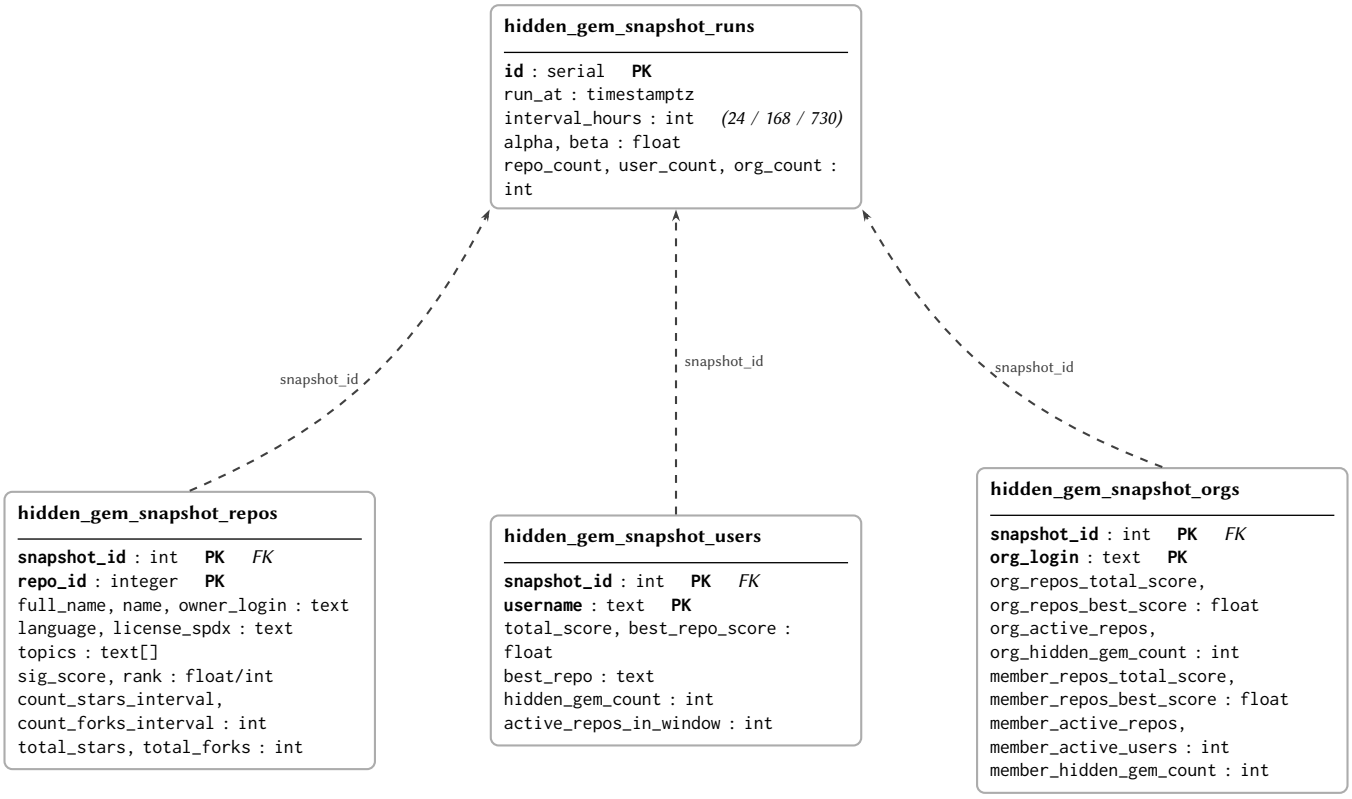

Figure 4: Hidden Gem snapshot tables. `hidden_gem_snapshot_runs` is the header record for each scoring run (one per interval per invocation); the three child tables store per-repo, per-user, and per-org significance scores for that run.

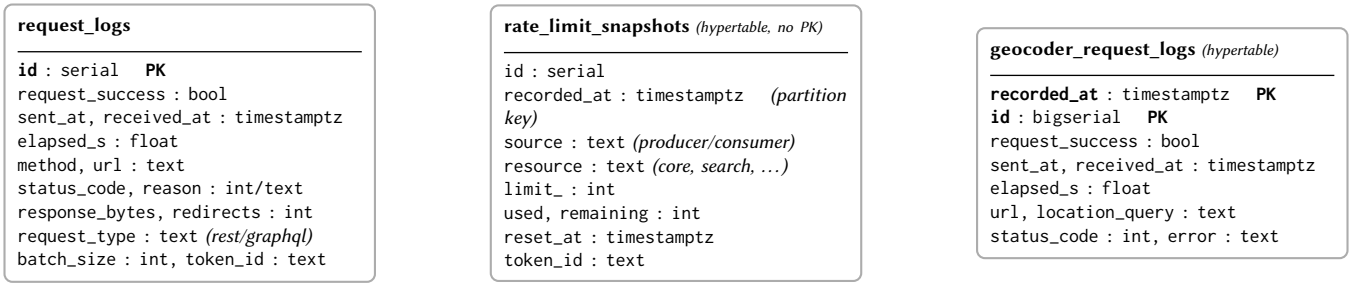

Figure 5: Operational log tables. These three tables record HTTP request logs for the GitHub REST/GraphQL API (`request_logs`), GitHub API rate-limit snapshots (`rate_limit_snapshots`), and Photon geocoder request logs (`geocoder_request_logs`). They carry no foreign key references to the core schema and are used exclusively for operational monitoring.

D Deployment Quick Reference

The full setup instructions, environment variable reference, and operational runbook are maintained in the project repository and kept up to date alongside the codebase:

https://github.com/BDP26/pm4-github-insights

The repository `README.md` covers prerequisites, Docker Compose profiles, and service URLs. `docs/RUNBOOK.md` and `docs/ENV.md` document operational procedures and environment variable configuration respectively.